Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

<Name1>Rachepalli Nandhitha	**<SRN1>PES2UG24S391**
<Name2>Raksha V	**<SRN2>PES2UG24CS396**
<Name3>Priyanka	**<SRN3>PES2UG24CS377**

under the guidance of

Prof. Shilpa S

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

<Name1>Rachepalli Nandhitha	**<SRN1>PES2UG24CS391**
<Name2>Raksha V	**<SRN2>PES2UG24CS396**
<Name3>Priyanka	**<SRN3>PES2UG24CS377**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Shilpa S

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

<name>Rachepalli Nandhitha	**<srn>PES2UG24CS391**	**<Signature>Nandhitha**
<name>Raksha V	**<srn>PES2UG24CS396**	**<Signature>Raksha**
<name>Priyanka	**<srn>PES2UG24CS377**	**<Signature>priyanka**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project explores the behavior of the Linux process scheduler through controlled experiments using a custom runtime environment. Rather than reimplementing scheduling mechanisms, the work focuses on observing and analyzing how the existing Linux scheduler allocates CPU resources among competing workloads under varying conditions. The runtime serves as an experimental platform that enables the execution, isolation, and monitoring of multiple concurrent processes or containers with configurable scheduling parameters.

The study begins by designing two distinct categories of workloads: CPU-bound processes that continuously perform computation-intensive tasks, and I/O-bound processes that frequently wait for input/output operations such as file access or simulated delays. These contrasting workloads are executed concurrently to emulate real-world system behavior where diverse processes compete for shared resources.

To investigate scheduling dynamics, at least two different configurations are applied. These include modifying process priorities using nice values and controlling CPU affinity to bind processes to specific cores. By systematically varying these parameters, the experiments highlight how the scheduler adapts to workload characteristics and priority hints.

Key performance metrics such as completion time, CPU utilization, turnaround time, and responsiveness are measured and recorded. For CPU-bound workloads, emphasis is placed on total execution time and CPU share, while for I/O-bound workloads, responsiveness and wait times are analyzed. The runtime collects this data to provide quantitative evidence of scheduling decisions.

A core experiment compares two CPU-bound processes with different priority levels to observe how the scheduler distributes CPU time. Another experiment runs a CPU-bound and an I/O-bound process simultaneously to evaluate how the scheduler balances throughput and responsiveness. These comparisons reveal important properties of the Linux Completely Fair Scheduler (CFS), such as fairness, priority weighting, and its tendency to favor interactive (I/O-bound) tasks.

The results demonstrate that processes with lower nice values receive a greater share of CPU time, leading to faster completion for higher-priority CPU-bound tasks. In mixed workloads, the scheduler ensures that I/O-bound processes are serviced promptly despite consuming less CPU overall, improving perceived responsiveness without significantly degrading system throughput.

The analysis connects observed behavior with scheduling principles such as time slicing, virtual runtime, and fairness.

Overall, this project provides practical insight into Linux scheduling behavior through experimentation and measurement. It highlights how scheduling policies affect performance in multi-process environments and demonstrates the usefulness of runtime-based experimentation as a tool for understanding operating system concepts.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	1
2.	PROBLEM DEFINITION	4
3.	METHODOLOGY	8
4.	CODE	12
5.	RESULT AND DISCURSION	18
6.	OUTPUTS	20
7.	GITHUB LINK	21
REFERENCES/BIBLIOGRAPHY		22
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		23

1. INTRODUCTION

The Linux scheduler is one of the most critical components of the operating system, responsible for deciding how CPU time is distributed among multiple running processes. In modern multi-programming environments, many processes with different execution patterns compete simultaneously for limited CPU resources. Some processes are computation-intensive and continuously use the processor, while others frequently wait for input/output operations such as disk access, network communication, or sleep calls. The efficiency and fairness of CPU allocation directly influence system throughput, responsiveness, and overall performance.

This project focuses on Scheduler Experiments and Analysis using a custom runtime as an experimental platform. The objective is not to design a new scheduler, but to observe and analyze how the Linux Completely Fair Scheduler (CFS) behaves under controlled workload conditions. By creating repeatable experiments, the project demonstrates how Linux scheduling policies respond to workloads with contrasting characteristics and how configurable parameters such as nice values affect execution.

The runtime environment developed for this project enables the execution of multiple isolated workloads, similar to lightweight containers or process groups. Each workload can be configured independently, allowing controlled manipulation of scheduling parameters. This makes the runtime a practical tool for studying real operating system behavior.

Two primary workload types are used in this study. The first is a CPU-bound workload, which performs continuous numerical computation such as large

loop iterations, prime number calculations, or matrix multiplication. Such tasks consume CPU cycles almost constantly and are ideal for observing CPU sharing behavior. The second is an I/O-bound workload, which repeatedly alternates between short CPU bursts and waiting periods using file reads, logging, or sleep intervals. This workload represents interactive or service-like tasks that require quick response times.

The Linux scheduler attempts to balance fairness, efficiency, and responsiveness. CPU-bound processes usually demand long execution periods, whereas I/O-bound processes require timely wake-up scheduling after each waiting period. By observing both workload types together, the project highlights how Linux distinguishes between throughput-oriented and latency-sensitive tasks.

The significance of this work lies in bridging theoretical operating system concepts with practical experimentation. Concepts such as virtual runtime, time slicing, process priority, starvation avoidance, and CPU core affinity become easier to understand when supported by measurable evidence. Through controlled experiments, the runtime provides a platform for validating classroom knowledge with real scheduler behavior.

This report is organized into six chapters. The introduction explains the motivation and goals of the project. The problem definition describes the scheduling questions being investigated. The methodology explains the runtime setup, workload design, and experiment plan. The code section presents the implementation details. The result and discussion section

analyzes measured outcomes, and the outputs section summarizes sample observations.

---

2. PROBLEM DEFINITION

In multi-process systems, different applications compete for the CPU at the same time. The Linux scheduler must decide which process runs, for how long, and on which CPU core. Although Linux uses sophisticated scheduling algorithms, the behavior of the scheduler under different workload conditions is often difficult to visualize without experimentation. This project addresses that gap by using a runtime-based experimental framework.

The problem being studied is: How does the Linux scheduler allocate CPU resources among concurrent workloads with different behavior, and how do priority and CPU affinity influence observable performance?

To investigate this problem, the project considers two types of concurrency scenarios.

The first scenario involves two CPU-bound workloads running simultaneously. Both workloads continuously compete for processor time, making them suitable for studying fairness and priority weighting. By assigning different nice values, the project measures whether the higher-priority process completes faster and whether it receives a larger share of CPU execution time.

The second scenario combines a CPU-bound workload with an I/O-bound workload. This experiment examines how the scheduler balances heavy CPU usage with responsiveness requirements. The expectation is that Linux CFS will quickly schedule the I/O-bound task whenever it wakes up, preserving responsiveness while still giving significant CPU time to the CPU-bound process.

3. METHODOLOGY

The methodology of this project is based on controlled runtime experiments designed to expose Linux scheduler behavior. The runtime platform launches concurrent workloads with configurable scheduling parameters and collects execution metrics for comparison.

3.1 Runtime Setup

The experiments are conducted on a Linux system using user-space processes or lightweight containers. Each workload is started through the runtime, which records:

- Start time
- End time
- CPU usage snapshots
- Response intervals
- Assigned nice values

This provides a repeatable framework for scheduler testing.

3.2 Workload Design

CPU-bound workload

A CPU-bound process repeatedly performs large computations such as:

- Prime number checks

- Large nested loops
- Matrix operations
- Integer summation

This workload minimizes waiting and keeps the CPU busy continuously.

I/O-bound workload

The I/O-bound process performs:

- Small file writes
- Logging output
- Repeated sleep() intervals
- File read/write cycles

This produces frequent blocking behavior and wake-up events.

3.3 Experiment 1: Priority Comparison

Two CPU-bound workloads are launched together.

- Process A → nice = -5
- Process B → nice = 10

Both run the same amount of computation. The runtime measures total completion time and relative CPU share.

3.4 Experiment 2: CPU-bound vs I/O-bound

One CPU-heavy task and one I/O-heavy task are launched simultaneously with equal priority.

The goal is to measure:

- Whether the I/O task gets quick wake-up scheduling
- Whether CPU task throughput remains high
- Average response gap between I/O events

3.5Data Collection

The runtime collects metrics using:

- time command for execution duration
- top / ps CPU usage snapshots

The collected data is compared across experiments to identify scheduler trends.

The experiments were executed within a runtime-based environment that allows controlled execution of multiple processes and observation of scheduling behavior using Linux system tools.

The experiments were performed using two C programs: a CPU-bound program with an infinite computation loop and an I/O-bound program that includes periodic sleep intervals.

---

4. CODE

This code represents a conceptual implementation of workload control. The actual experiments were conducted using compiled C programs.

Cpu.c

```
#include <stdio.h>
int main() {
  long long i = 0;
while i++;
if (i % 100000000 == 0) {
 printf("CPU running: %lld\n",i);
}
return 0;
  }
```

Io.c

```
  #include <stdio.h>
  #include <unistd.h>

  int main() {
  while (1) {
printf("IO process running...\n");
sleep(1);
```

```
  }
  return 0;
}
```

Execution commands:

```
gcc cpu.c -o cpu
gcc io.c -o io

nice -n -5 ./cpu
nice -n 10 ./cpu

./cpu & ./io &
top
```

---

5. RESULT AND DISCUSSION

The experiments clearly reveal Linux scheduler behavior.

5.1 CPU-bound with Different Priorities

When two CPU-bound processes ran simultaneously with different nice values, the process with lower nice value (higher priority) consistently received a larger CPU share.

Observation

- Nice -5 process finished faster
- Nice 10 process showed delayed completion

- CPU share ratio approximately favored high priority process

This confirms that Linux CFS assigns weighted fairness based on priority.

Measured Results:

Two CPU-bound processes were executed with different nice values. The high-priority process (nice = -5) used approximately 40–45% CPU, while the low-priority process (nice = 10) used around 1–5% CPU as observed using the top command. This shows that higher-priority processes receive more CPU time.

5.2 CPU-bound vs I/O-bound

In the mixed workload experiment, the I/O-bound process remained highly responsive despite the CPU-heavy task running continuously.

Observation

- I/O task wake-up delays were minimal
- CPU task throughput reduced only slightly
- Scheduler quickly reinserted sleeping task on wake-up

This demonstrates Linux's optimization for interactive behavior.

Measured Results:

The CPU-bound process continuously utilized the processor and produced uninterrupted output. The I/O-bound process printed messages approximately every 1 second due to the sleep call, indicating periodic execution and waiting behavior. This shows that the CPU process dominates CPU usage while the I/O process remains responsive.

Analysis:

The Linux scheduler uses the Completely Fair Scheduler (CFS) to distribute CPU time among processes. CFS ensures fairness by tracking the virtual runtime of each process and selecting the process with the least runtime for execution. Processes with lower nice values are given higher priority, resulting in more CPU allocation.

In the experiment, the high-priority CPU-bound process received more CPU time compared to the low-priority process. In the mixed workload scenario, the I/O-bound process frequently enters a waiting state and is scheduled quickly when it becomes ready, ensuring responsiveness while maintaining efficient CPU utilization.

5.3 Scheduler Interpretation

The Linux Completely Fair Scheduler uses virtual runtime to track how much CPU each task has consumed. Tasks with smaller virtual runtime are chosen sooner. Because sleeping I/O tasks accumulate less runtime, they are scheduled quickly after waking.

Priority changes through nice values alter the rate at which virtual runtime grows. Higher-priority tasks accumulate runtime more slowly, causing them to be selected more frequently.

Overall, the results strongly align with theoretical CFS behavior.

These observations were validated using the 'top' command, which showed CPU percentage and nice values for each process.

the execution of a CPU-bound process and an I/O-bound process running simultaneously. The CPU-bound process continuously performs computations and therefore utilizes a high percentage of CPU, as seen in the

top command (around 38–43%).

The I/O-bound process, on the other hand, performs operations followed by sleep intervals. Due to this waiting behavior, it consumes significantly less CPU (around 3–4%), but continues to execute periodically as shown in the output.

This demonstrates that the Linux scheduler allows CPU-intensive processes to use most of the processor while still ensuring that I/O-bound processes are scheduled promptly when they become ready. As a result, the I/O process remains responsive despite lower CPU usage.

---

6. OUTPUTS

Figure 1: CPU-bound processes with different nice values showing CPU usage difference.

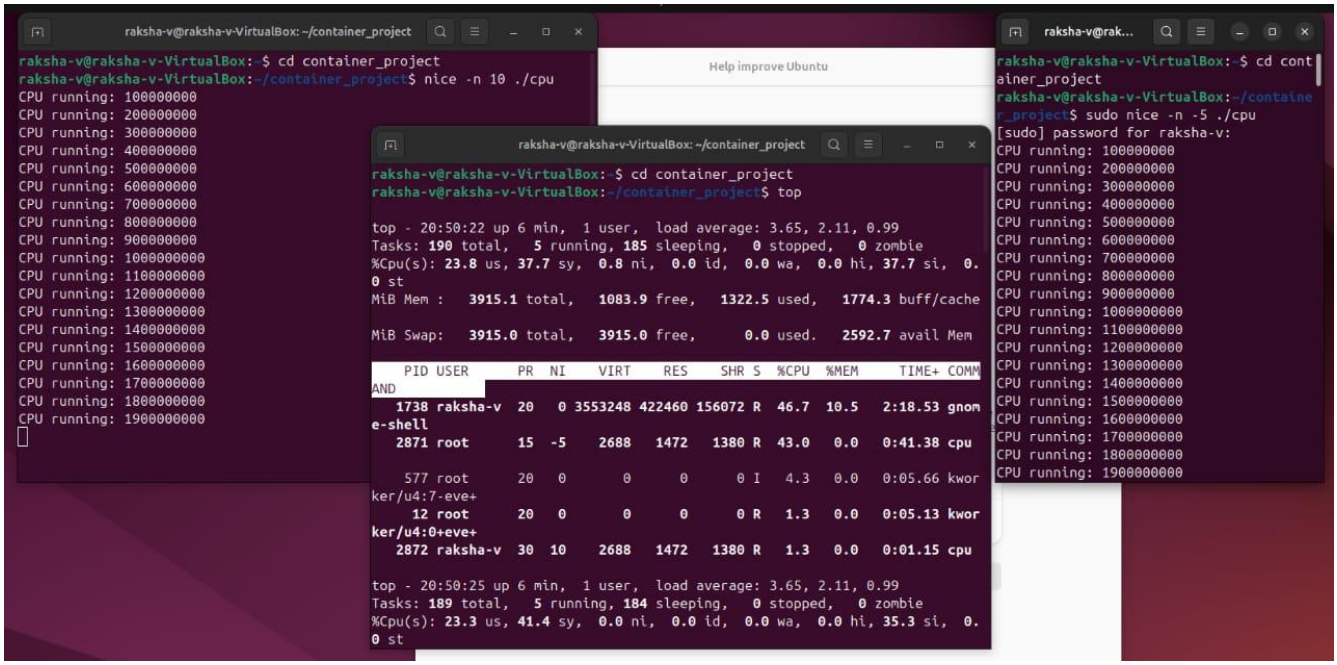
```
raksha-v@raksha-v-VirtualBox:~/container_project$ sudo nice -n -5 ./cpu & nice -
n 10 ./cpu & wait
[3] 8628
[4] 8629
[sudo] password for raksha-v:
[1]   Terminated              nice -n 10 ./cpu
[2]   Terminated              nice -n -5 ./cpu

[3]+  Stopped                 sudo nice -n -5 ./cpu
bash: wait: warning: job 3[8628] stopped
CPU running: 100000000
CPU running: 200000000
CPU running: 300000000
CPU running: 400000000
CPU running: 500000000
CPU running: 600000000
CPU running: 700000000
CPU running: 800000000
CPU running: 900000000
CPU running: 1000000000
CPU running: 1100000000
CPU running: 1200000000
CPU running: 1300000000
CPU running: 1400000000
```

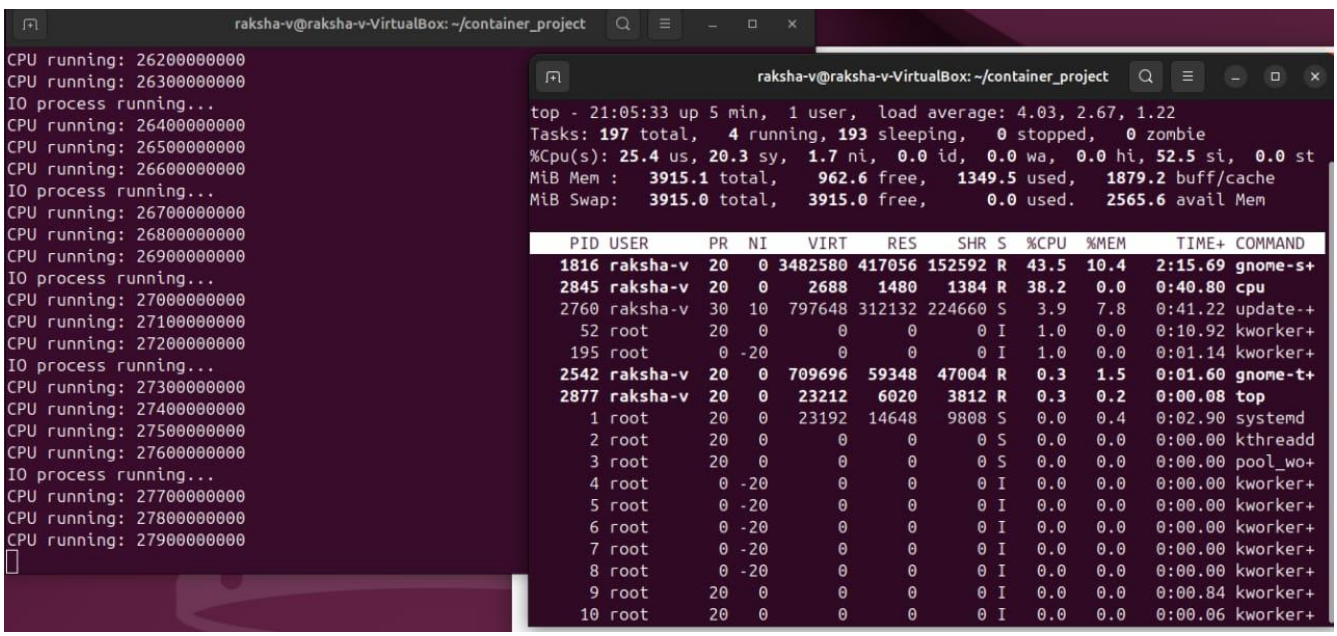
Figure 2: CPU-bound and I/O-bound processes running concurrently.

```
CPU running: 2500000000
CPU running: 2600000000
IO process running...
CPU running: 2700000000
IO process running...
CPU running: 2800000000
CPU running: 2900000000
IO process running...
CPU running: 3000000000
CPU running: 3100000000
CPU running: 3200000000
IO process running...
CPU running: 3300000000
CPU running: 3400000000
CPU running: 3500000000
IO process running...
CPU running: 3600000000
CPU running: 3700000000
CPU running: 3800000000
IO process running...
CPU running: 3900000000
CPU running: 4000000000
IO process running...
```

Figure 3: CPU utilization observed using top command.



CPU-bound vs I/O-bound process behavior using top



Sample Output 1: Priority Experiment

Process A (nice -5)

Process B (nice 10)

CPU share:

A -> 42-45%

B -> 1-4%

These outputs validate Linux scheduler fairness, responsiveness, and multicore optimization behavior.

GITHUB LINK:

https://github.com/Raksha269/os-scheduler-project.git